

Senior Flutter Interview Master Guide

Candidate: Irshath Ahamed | Senior Flutter Developer (Portal/Mobile)

Senior Flutter Developer (Portal / Mobile) — Interview Master Guide

Target Role: Senior Flutter Developer (Portal/Mobile) | Friday 10:00 AM

Candidate: Irshath Ahamed (~5 Years Production Experience)

Table of Contents

- [1. The 90-Second Elevator Pitch & Personal Narrative](#)
- [2. Deep Dive on Past Projects \(Speaking With Depth\).](#)
 - Fusion Electro Mechanical (Offline-First Enterprise Portal & Mobile Platform)*
 - Golden Hippo Technology (White-Label 1,000+ App Platform & Build Automation)*
- [3. Flutter & Dart Internals \(The Architecture & Engine Round\).](#)
 - Three Trees: Widget, Element, and RenderObject
 - Dart Concurrency, Event Loop, Microtasks & Isolates
 - Memory Management & Generational Garbage Collection
 - Keys (`ValueKey` , `ObjectKey` , `GlobalKey` , `PageStorageKey`)
- [4. State Management & Clean Architecture](#)
 - Clean Architecture Layers (Domain, Data, Presentation)
 - BLoC / Cubit in Enterprise Production (Event transformations, memory management)
 - BLoC vs Riverpod vs Provider vs GetX (Trade-offs & when to use what)
 - Working in Existing/Legacy Codebases without Regressions
- [5. Portal, Tablet & Responsive Engineering \(The Core Job Requirement\).](#)
 - Mobile vs Tablet vs Web Portal Form Factors
 - Dynamic Layouts: `LayoutBuilder` , `MediaQuery.sizeOf(context)`
 - Master-Detail Navigation, Shell Routes (`go_router`)
 - Mouse Cursor, Hover States, Keyboard Shortcuts & Web Constraints
- [6. Offline-First, Data Sync & Local Storage](#)
 - Hive vs SQLite (sqflite / Drift) vs ObjectBox
 - Local Outbox Sync Pattern & Optimistic UI
 - Conflict Resolution Strategies (Server-Wins vs LWW vs Field-Level Merging)
 - Secure REST Interceptors, Token Refresh Queueing
- [7. Performance Optimization, Memory & Profiling](#)
 - Flutter DevTools (CPU Profiler, Memory Allocator, Network Tab)
 - Diagnosing & Fixing Memory Leaks (Retaining Paths)
 - Rendering Performance: Repaint Boundaries, Skia/Impeller Shader Warmup

- App Startup Time Optimization
 - 8. [Automated Testing Strategy \(Unit, Widget, Integration\)](#)
 - BLoC Unit Testing (`bloc_test` , `mocktail`)
 - Widget Testing & Golden Tests
 - Integration Testing on Multi-Platform Surfaces
 - 9. [Senior Live Coding & Dart Coding Challenges](#)
 - Challenge 1: Custom Debounce & Throttle Utility
 - Challenge 2: Production Infinite-Scroll Pagination Cubit
 - Challenge 3: In-Memory LRU Cache with TTL
 - Challenge 4: Heavy Background JSON Parsing with Isolates
 - 10. [Senior Behavioral, Cross-Functional & Scenario Questions](#)
 - Handling Technical Debt & Legacy Code
 - Cross-Functional Conflicts (Backend API disputes, QA blockers)
 - Production Outage & Crash Triage Protocols
 - 11. [Reverse Questions to Ask the Interviewers](#)
-

1. The 90-Second Elevator Pitch & Personal Narrative

The Script (Tailored to the Job Description)

"Hi, thank you for having me today. I'm Irshath Ahamed, a Senior Flutter Developer with close to five years of hands-on experience architecting, scaling, and maintaining production cross-platform applications across both mobile (Android/iOS) and portal/tablet form factors.

Most recently at Fusion Electro Mechanical in Abu Dhabi, I architected and built an enterprise offline-first field workforce platform completely from scratch. What makes this especially relevant to your role is that it is a unified codebase serving field workers on mobile devices while simultaneously powering a dense, responsive admin and management portal for tablet and web. I designed it from day one using Clean Architecture, BLoC/Cubit, Hive for offline-first local persistence with bi-directional Supabase sync, and robust timesheet/compliance automation.

Prior to that at Golden Hippo, I spent nearly three years maintaining, debugging, and extending an established, live production enterprise platform delivering 1,000+ client-branded applications from a single shared codebase using Flutter Flavors, and built build automation pipelines that shaved release cycle times by 30%.

So I bring both sides of the coin: the architectural vision to design robust systems from scratch, and the practical engineering discipline to jump into an existing live production codebase—extending features, eliminating technical debt, designing responsive mobile-to-portal interfaces, and establishing rock-solid stability with testing. That is exactly where my core expertise lies, and I'm very excited about this discussion."

2. Deep Dive on Past Projects (Speaking With Depth)

The job description explicitly states:

"Able to speak fluently and in detail about own past Flutter projects (architecture choices, trade-offs, problems solved) - candidates must demonstrate depth, not just list technologies."

Project 1: Fusion Enterprise Field Workforce Platform

- **The Problem:** Hundreds of field engineers operate in basement utility shafts, remote construction sites, and marine facilities across Abu Dhabi with zero or intermittent cellular connectivity. Manual paper logs resulted in payroll discrepancies, delayed maintenance reports, and inaccurate timesheet tracking.
- **The Architecture:**
 - **Clean Architecture:** Strict separation of concerns:
 - **Data** : Hive local boxes, Supabase PostgreSQL with RLS, Firebase Auth.
 - **Domain** : Pure Dart Entities (`AttendanceRecord` , `EmployeeEntity`), Repository interfaces, and domain calculators (`TimesheetCalculator` , `SalaryCycle`).
 - **Presentation** : BLoC/Cubit (`AttendanceCubit` , `UserManagementCubit` , `TimesheetCubit`), responsive screen widgets.
 - **Portal vs Mobile Unification:**
 - Built a single Flutter codebase that compiles to Android/iOS mobile apps and a high-density Web/Tablet Portal.
 - Designed adaptive layouts using `LayoutBuilder` and breakpoints:
 - `< 768px` : Bottom navigation bar, vertical cards, full-screen dialogs, camera capture.
 - `768px – 1100px` (Tablet): Collapsible navigation rail, split Master-Detail timesheet inspector.
 - `> 1100px` (Desktop Portal): Fixed sidebar, multi-column analytics, inline timesheet audit tables, data export modals.
- **Key Technical Challenges & How You Solved Them:**
 1. **Offline-First Synchronization & Conflict Resolution:**
 - *Issue:* Records generated offline must not block user operations, but cannot create ID collisions when pushed to Supabase.
 - *Solution:* Generated client-side UUID v4 keys instantly. Records are saved locally in Hive with an `isSynced = false` flag. A background sync worker detects connectivity changes via `connectivity_plus`, pulls server changes, and pushes pending local logs using an idempotency key to prevent duplicate check-ins.
 2. **Memory Leak / Out of Memory (OOM) on Web Portal During Dense Timesheet Auditing:**
 - *Issue:* Loading 2,000+ employee records with base64 facial recognition thumbnails caused Flutter Web heap crashes.
 - *Solution:* Implemented an in-memory decoded image cache with LRU eviction and memory bounds (`PaintingBinding.instance.imageCache.maximumSizeBytes`), converted unneeded base64 previews to lazy image memory loaders, and implemented list virtualization so only visible DOM nodes are rendered.
 3. **Complex Business Logic (Auto-Checkout & Shift Capping):**
 - *Issue:* Workers occasionally forgot to check out at the end of a shift, which previously ran open timers for 48+ hours and inflated overtime.
 - *Solution:* Created `TimesheetCalculator.calculateDailyTimesheets()`. If a shift exceeds 24 hours without checkout, the system automatically caps the regular hours at 8.0, flags

`isAutoCompleted = true`, and sets overtime to 0.0, alerting the administrator on the portal to review and override.

Project 2: Golden Hippo (White-Label Platform for 1,000+ Brands)

- **The Problem:** Maintaining separate codebases or branches for hundreds of client brands was impossible to scale and led to merge hell and inconsistent bug fixes.
- **The Solution:**
 - Maintained a single core codebase using **Flutter Flavors** (`flavor: brandA`, `flavor: brandB`).
 - Extracted brand-specific configurations (theme colors, typography, logos, API base URLs, payment gateway credentials) into dedicated JSON/Dart config schemas.
 - Wrote Node.js automation scripts integrated with AWS S3 that dynamically inject asset bundles, compile the target flavor, and upload completed binaries with hash verification, cutting release turnaround by 30%.
- **Architecture Trade-Off:**
 - *Trade-Off:* Using compile-time flavors vs runtime remote config. Compile-time flavors increased build pipeline complexity, but guaranteed zero bundle bloat for end users (no assets of Brand B included in Brand A's APK) and ensured security compliance.

3. Flutter & Dart Internals (The Architecture & Engine Round)

The Three Trees Architecture

Widget Tree (Immutable Config) → Element Tree (Structural Lifecycle) → RenderObject Tree (Layout)

- **Widget:** Immutable configuration. Cheap to instantiate and recreate frequently during `build()`.
- **Element:** Mutable bridge. Represents an instantiated widget in a specific location in the tree. Manages lifecycle, state retention, and dirty marking (`markNeedsBuild()`).
- **RenderObject:** Handles layout, sizing, hit-testing, and actual painting on the Skia/Impeller canvas. Expensive to create.
- **Reconciliation Algorithm:** When a parent widget rebuilds, Flutter compares the new widget with the existing element:

```
Widget.canUpdate(Widget oldWidget, Widget newWidget) {  
    return oldWidget.runtimeType == newWidget.runtimeType && oldWidget.key == newWidget.key;  
}
```

- If `true`: The Element retains its identity and updates its widget reference; the RenderObject is updated without being re-created.
- If `false`: The entire old element subtree and its RenderObjects are unmounted and disposed, and a new element tree is mounted.

Keys in Flutter

Key Type	Use Case
<code>ValueKey<T></code>	Used when items have a natural, unique scalar identifier (e.g., <code>ValueKey(employee.id)</code> in a reorderable list).
<code>ObjectKey</code>	Compares by object reference/identity rather than scalar value.
<code>UniqueKey()</code>	Forces the creation of a new Element and RenderObject on every build.
<code>PageStorageKey</code>	Preserves scroll position across tab switches or navigator changes.
<code>GlobalKey</code>	Grants access to an Element/State across different subtrees (e.g., <code>FormState</code> , scaffold drawers). Warning: Expensive because it forces subtree reparenting and traverses the entire tree.

Dart Concurrency: Event Loop, Microtasks & Isolates

- Dart is **single-threaded by default** with an **Event Loop**:

```
[Microtask Queue (Higher Priority)] —> [Event Queue (I/O, Timers, Gestures, Network)]
```

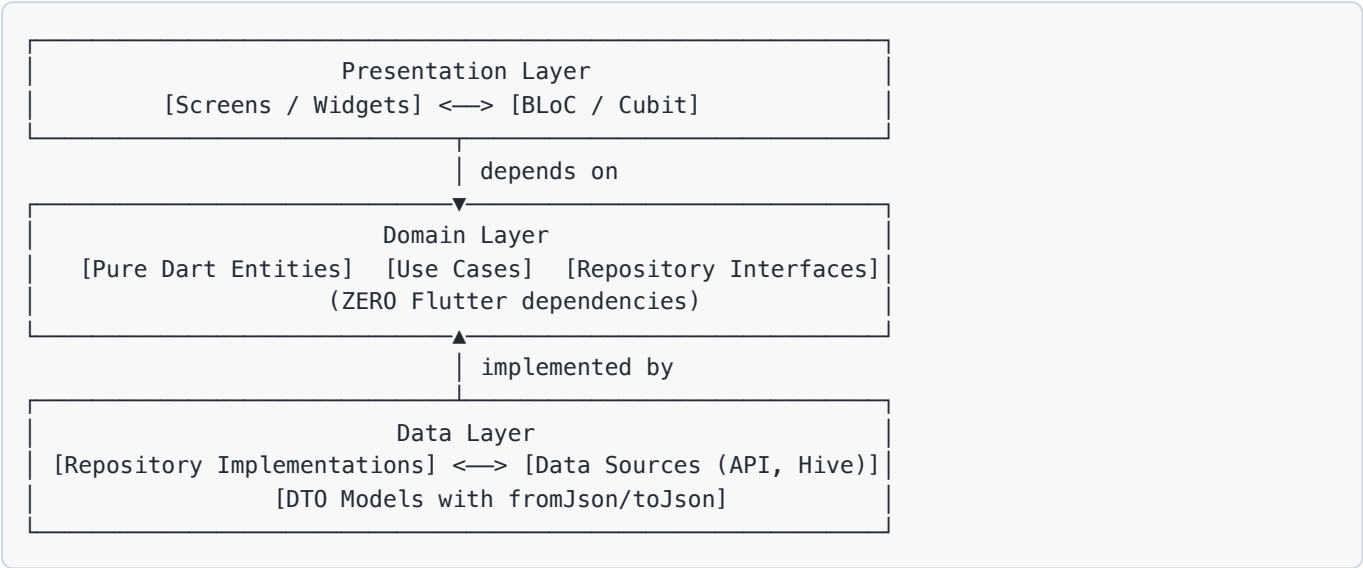
- Microtasks:** Run before the next event from the Event Queue. Created via `scheduleMicrotask()`.
- Event Queue:** Handles external events like user touch, HTTP responses, file I/O, and `Timer.run()`.
- Isolates:**
 - True multi-threading in Dart. Each Isolate has its **own dedicated memory heap** and its own event loop (no shared mutable memory, eliminating race conditions and deadlocks).
 - Communication happens exclusively via message passing (`SendPort` and `ReceivePort`).
 - Use `Isolate.run()` or `compute()` for heavy tasks (e.g., parsing a 5MB JSON response or resizing images) to prevent dropping frames (jank) on the UI thread (target: 60fps/120fps = 16.6ms / 8.3ms per frame).

Dart Memory Management & Garbage Collection

- Generational Garbage Collection:**
 - Young Generation (Scavenger):**
 - Optimized for short-lived objects (e.g., ephemeral widgets created during `build()`).
 - Uses a copying collector (semi-space). Surviving objects after a cycle are moved to the Old Generation.
 - Very fast (usually runs in 1–2 milliseconds).
 - Old Generation (Mark-Sweep / Mark-Compact):**
 - For long-lived objects (e.g., singletons, repositories, cached images, controllers).
 - Traverses pointer references, marks reachable objects, sweeps dead memory, and compacts fragmented blocks.
-

4. State Management & Clean Architecture

Clean Architecture Layers



BLoC vs Cubit

- **Cubit**: Method-driven (`cubit.fetchEmployees()`). Simple, concise, ideal for standard CRUD, navigation, and form state where complex event transformations are unnecessary.
- **BLoC**: Event-driven (`bloc.add(FetchEmployeesEvent())`). Indispensable when you need advanced stream transformations:
 - **Debouncing**: Search inputs (wait 300ms after user stops typing).
 - **Dropping / Throttling**: Preventing double-taps on payment or check-in buttons using `droppable()` from `bloc_concurrency`.
 - **Switching**: Cancelling in-flight requests when a new filter is selected using `restartable()`.

State Management Comparison Matrix

Framework	Strengths	Trade-offs / Weaknesses	Best Fit
BLoC / Cubit	Strict separation, predictable state machine, outstanding testing tooling (<code>bloc_test</code>), industry standard for enterprise.	Boilerplate events/states; requires discipline.	Enterprise applications, data-heavy apps, banking, team environments.
Riverpod	Compile-time safe, no <code>BuildContext</code> required, autodispose, family providers, zero boilerplate.	Steeper learning curve for junior developers; API evolution across v1/v2.	Modern apps, reactive real-time caching, complex dependency graphs.
Provider	Lightweight, officially recommended by Flutter team for simpler apps, low barrier to entry.	Runtime errors if provider is missing above context; no built-in event queuing.	Small-to-medium apps, simple shared state.
GetX	Fast prototyping, no context needed for routing/snackbars.	Violates Flutter idiomatic patterns; global static state makes unit testing and Clean Architecture difficult.	Prototypes, rapid MVPs (avoid in large enterprise codebases).

Working in Existing/Legacy Codebases (Without Regressions)

When interviewing for a role that emphasizes *enhancing an existing application*:

- 1. **Never do massive "big-bang" rewrites:**
 - Use the **Strangler Fig Pattern**: Incrementally replace legacy modules behind stable interfaces.
- 2. **Isolate with Abstractions:**
 - Create a repository contract (`abstract class EmployeeRepository`) between the legacy data source and new features.
- 3. **Guard with Tests Before Refactoring:**
 - Write characterization / unit tests around the existing behavior to lock down current expectations before touching any business logic.
- 4. **Feature Flags:**
 - Wrap newly enhanced modules with remote or local feature toggles to enable instant rollback if edge cases emerge in production.

5. Portal, Tablet & Responsive Engineering (The Core Job Requirement)

The Job Description highlights:

"Hands-on production experience building Flutter interfaces for portal/tablet form factors specifically — mobile-only experience is not sufficient for this role."

Key Differences: Mobile vs Portal/Tablet

Aspect	Mobile (Phone)	Portal / Tablet (Desktop/Web)
Aspect Ratio & Screen Space	Vertical (9:16), compact (360-430dp wide)	Horizontal (16:9 / 4:3), spacious (768dp – 1920dp+)
Input Modality	Touch, gestures, virtual keyboard	Mouse hover, right-click, scroll wheel, physical keyboard
Navigation Pattern	Stack-based (<code>push</code> , <code>pop</code>), bottom navigation bar	Master-Detail, persistent collapsible sidebar, tabs, breadcrumbs
Data Density	Single column cards, accordion lists, paginated carousels	Data grids, multi-column tables, inline sorting, side-by-side split panels
Dialogs / Modals	Full-screen routes or bottom sheets	Centered modal dialogs with fixed max-width (<code>constraints: BoxConstraints(maxWidth: 600)</code>)

Responsive Implementation Code Patterns

1. Responsive Breakpoints Builder

```

class ResponsiveLayout extends StatelessWidget {
  final Widget mobile;
  final Widget? tablet;
  final Widget portal;

  const ResponsiveLayout({
    super.key,
    required this.mobile,
    this.tablet,
    required this.portal,
  });

  static bool isMobile(BuildContext context) =>
    MediaQuery.sizeOf(context).width < 768;

  static bool isTablet(BuildContext context) =>
    MediaQuery.sizeOf(context).width >= 768 &&
    MediaQuery.sizeOf(context).width < 1100;

  static bool isPortal(BuildContext context) =>
    MediaQuery.sizeOf(context).width >= 1100;

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        if (constraints.maxWidth >= 1100) {
          return portal;
        } else if (constraints.maxWidth >= 768) {
          return tablet ?? portal;
        } else {
          return mobile;
        }
      },
    );
  }
}

```

Crucial Senior Tip: Mention using `MediaQuery.sizeOf(context)` (introduced in Flutter 3.10) instead of `MediaQuery.of(context).size`. The older method registers a dependency on *all* MediaQuery properties (insets, padding, orientation, brightness), causing unnecessary rebuilds of the entire screen when the soft keyboard opens or closes.

2. Adaptive Master-Detail View

```

Widget build(BuildContext context) {
  final isWide = MediaQuery.sizeOf(context).width >= 900;

  if (!isWide) {
    // Mobile Flow: List pushes to Detail Screen
    return EmployeeListView(
      onSelected: (emp) => Navigator.push(
        context,
        MaterialPageRoute(builder: (_) => EmployeeDetailScreen(employee: emp)),
      ),
    );
  }

  // Tablet/Portal Flow: Side-by-side Split View
  return Row(
    children: [
      SizedBox(
        width: 380,
        child: EmployeeListView(
          selectedId: _selectedEmployee?.id,
          onSelected: (emp) => setState(() => _selectedEmployee = emp),
        ),
      ),
      const VerticalDivider(width: 1),
      Expanded(
        child: _selectedEmployee != null
          ? EmployeeDetailPane(employee: _selectedEmployee!)
          : const Center(child: Text('Select an employee to view records')),
      ),
    ],
  );
}

```

3. Mouse & Pointer Interactions (Web/Portal Polish)

- Wrap clickable components with `MouseRegion(cursor: SystemMouseCursors.click)` so desktop/web users get expected pointer feedback.
 - Use `InkWell` or `FocusableActionDetector` to support keyboard navigation (`Tab` , `Enter` , `Escape`).
-

6. Offline-First, Data Sync & Local Storage

Local Storage Technologies Comparison

Storage	Type	Read/Write Speed	Complex Queries	Best Used For
Hive	NoSQL Key-Value / Binary Box	Blazing fast (in-memory indexed binary files)	Basic key lookup / filtering in memory	Caching entities, user tokens, app settings, offline queues.
sqlite / Drift	Relational SQLite	Fast	Excellent (joins, indexes, foreign keys, SQL)	Complex relational data, large multi-table datasets.
ObjectBox	High-performance NoSQL DB	Extremely fast	Good (built-in queries)	IoT, high-frequency sensor or GPS logging.

The Offline Outbox Sync Pattern

- 1. User Action:** The field engineer clocks in or logs emergency duty while offline.
- 2. Local Commit:**
 - Write record to local storage (Hive) immediately.
 - Assign a UUID client-side.
 - Mark `syncStatus = SyncStatus.pendingUpload`.
 - Update UI immediately (**Optimistic UI**).
- 3. Outbox Queue Worker:**
 - Connectivity listener triggers sync when connection is restored.
 - Batches pending records into a sync payload.
 - Pushes to backend REST / Supabase RPC endpoint.
 - On success: Marks records `syncStatus = SyncStatus.synced`.
 - On network failure: Implements **Exponential Backoff** with jitter (e.g., retry in 2s, 4s, 8s, 16s, up to 60s) to avoid overwhelming the server upon reconnect.

Conflict Resolution Strategies

- **Last-Write-Wins (LWW):** Compares UTC timestamps (`updatedAt`). Simple, but can overwrite intermediate changes.
- **Server-Authoritative:** Server overrides client state unless explicit local draft locks exist.
- **Field-Level / Three-Way Merging:** Compares base snapshot, client changes, and server changes; merges non-overlapping fields and flags conflicting fields for manual admin resolution.

Safe HTTP Token Refresh (Race Condition Prevention)

When multiple API requests trigger simultaneous 401 Unauthorized errors:

```

class AuthInterceptor extends QueuedInterceptor {
  final Dio dio;
  AuthInterceptor(this.dio);

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) async {
    if (err.response?.statusCode == 401) {
      // QueuedInterceptor automatically locks other in-flight requests!
      try {
        final newAccessToken = await refreshAuthToken();
        // Retry the original request with the new token
        err.requestOptions.headers['Authorization'] = 'Bearer $newAccessToken';
        final response = await dio.fetch(err.requestOptions);
        return handler.resolve(response);
      } catch (e) {
        // Refresh failed: log out user
        return handler.reject(err);
      }
    }
    handler.next(err);
  }
}

```

7. Performance Optimization, Memory & Profiling

Key DevTools Workflows

1. CPU Profiler & Performance View:

- Record trace while reproducing lag.
- Look for frames taking >16ms (jank).
- Trace flame chart down to expensive methods (e.g., excessive JSON decoding, un-memoized string parsing, or heavy calculations inside `build()`).

2. Memory View & Heap Snapshot:

- Capture Snapshot A (baseline).
- Perform user action (open timesheet audit modal, view photos).
- Pop screen back to root.
- Trigger GC (Garbage Collection).
- Capture Snapshot B.
- Compare Snapshot B against A. If controllers, screens, or image buffers are still alive, inspect the **Retaining Path** to find what is holding the reference.

Common Memory Leaks in Flutter

- **Uncanceled StreamSubscriptions & Timers:** Always cancel in `dispose()` or close Cubits/Blocs.
- **TextEditingController / ScrollControllers / AnimationControllers:** Must call `.dispose()` in State lifecycle.
- **Global Event Listeners / Singletons:** Registering a callback to a singleton (`connectivity.listen(...)`) without unregistering keeps the entire widget state in memory forever.

- **Large Image Caching:** Loading full 12MP camera photos into memory instead of downsampling with `ResizeImage` or caching decoded bytes.

Rendering Optimizations

- **Use `const` Constructors Everywhere:** Allows Flutter to short-circuit widget rebuilding.
- **`RepaintBoundary`:** Wrap independently animating widgets (e.g., a progress spinner, ticking clock, or live GPS marker) so the rest of the complex screen canvas does not repaint on every tick.
- **Avoid Expensive Canvas Operations:**
 - Replace `Opacity` widget (which introduces an expensive offscreen compositing buffer) with `Color.withValues(alpha: ...)` or `AnimatedOpacity` where possible.
 - Avoid unnecessary `ClipRect` inside scrolling list items; use container decoration border radii instead.

8. Automated Testing Strategy (Unit, Widget, Integration)

1. BLoC Unit Testing (`bloc_test`)

```
void main() {
  group('TimesheetCubit', () {
    late MockTimesheetRepository mockRepo;
    late TimesheetCubit cubit;

    setUp(() {
      mockRepo = MockTimesheetRepository();
      cubit = TimesheetCubit(repository: mockRepo);
    });

    tearDown(() => cubit.close());

    blocTest<TimesheetCubit, TimesheetState>(
      'emits [Loading, Loaded] when records are fetched successfully',
      build: () {
        when(() => mockRepo.getRecords(any()))
          .thenAnswer((_) async => [testRecord]);
        return cubit;
      },
      act: (cubit) => cubit.loadRecords('emp-101'),
      expect: () => [
        isA<TimesheetLoading>(),
        isA<TimesheetLoaded>().having(
          (s) => s.records.length,
          'records count',
          1,
        ),
      ],
      verify: (_) {
        verify(() => mockRepo.getRecords('emp-101')).called(1);
      },
    );
  });
}
```

2. Widget Testing

```
testWidgets('Renders export button and triggers callback on tap', (tester) async {
  bool exportTapped = false;

  await tester.pumpWidget(
    MaterialApp(
      home: Scaffold(
        body: ExportButton(onPressed: () => exportTapped = true),
      ),
    ),
  );

  expect(find.text('Export Timesheet'), findsOneWidget);
  await tester.tap(find.byType(ElevatedButton));
  await tester.pumpAndSettle();

  expect(exportTapped, isTrue);
});
```

9. Senior Live Coding & Dart Coding Challenges

Challenge 1: Custom Debounce & Throttle Utility

Often asked to test understanding of asynchronous programming and timers.

```
import 'dart:async';

class Debouncer {
  final Duration delay;
  Timer? _timer;

  Debouncer({required this.delay});

  void run(void Function() action) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }

  void dispose() => _timer?.cancel();
}

class Throttler {
  final Duration interval;
  Timer? _timer;
  bool _isThrottling = false;

  Throttler({required this.interval});

  void run(void Function() action) {
    if (!_isThrottling) {
      action();
      _isThrottling = true;
      _timer = Timer(interval, () => _isThrottling = false);
    }
  }

  void dispose() => _timer?.cancel();
}
```

Challenge 2: Production Infinite-Scroll Pagination Cubit

```
abstract class PaginatedState {}
class PaginatedInitial extends PaginatedState {}
class PaginatedLoading extends PaginatedState {
  final List<dynamic> oldItems;
  final bool isFirstFetch;
  PaginatedLoading(this.oldItems, {this.isFirstFetch = false});
}
class PaginatedLoaded extends PaginatedState {
  final List<dynamic> items;
  final bool hasReachedMax;
  PaginatedLoaded({required this.items, required this.hasReachedMax});
}
class PaginatedError extends PaginatedState {
  final String message;
  PaginatedError(this.message);
}

class PaginatedCubit extends Cubit<PaginatedState> {
  final Future<List<dynamic>> Function(int page, int limit) fetchPage;
  int _page = 1;
  static const int _limit = 20;

  PaginatedCubit({required this.fetchPage}) : super(PaginatedInitial());

  void loadItems() async {
    if (state is PaginatedLoading) return;

    final currentState = state;
    var oldItems = <dynamic>[];
    if (currentState is PaginatedLoaded) {
      if (currentState.hasReachedMax) return;
      oldItems = currentState.items;
    }

    emit(PaginatedLoading(oldItems, isFirstFetch: _page == 1));

    try {
      final newItems = await fetchPage(_page, _limit);
      _page++;
      final bool hasReachedMax = newItems.length < _limit;
      emit(PaginatedLoaded(
        items: oldItems + newItems,
        hasReachedMax: hasReachedMax,
      ));
    } catch (e) {
      emit(PaginatedError(e.toString()));
    }
  }

  void refresh() {
    _page = 1;
    loadItems();
  }
}
```

Challenge 3: In-Memory LRU Cache with TTL

```
class CacheItem<V> {
  final V value;
  final DateTime expiry;
  CacheItem(this.value, this.expiry);
  bool get isExpired => DateTime.now().isAfter(expiry);
}

class LruCache<K, V> {
  final int capacity;
  final Duration ttl;
  final Map<K, CacheItem<V>> _cache = {};

  LruCache({required this.capacity, required this.ttl});

  V? get(K key) {
    final item = _cache[key];
    if (item == null) return null;
    if (item.isExpired) {
      _cache.remove(key);
      return null;
    }
    // Refresh LRU order by re-inserting
    _cache.remove(key);
    _cache[key] = item;
    return item.value;
  }

  void put(K key, V value) {
    if (_cache.containsKey(key)) {
      _cache.remove(key);
    } else if (_cache.length >= capacity) {
      // Remove least recently used (first key in LinkedHashMap)
      final oldestKey = _cache.keys.first;
      _cache.remove(oldestKey);
    }
    _cache[key] = CacheItem(value, DateTime.now().add(ttl));
  }
}
```

Challenge 4: Heavy Background JSON Parsing with Isolates

```
import 'dart:convert';
import 'dart:isolate';

Future<List<EmployeeEntity>> parseLargeEmployeePayload(String rawJson) async {
  // Isolate.run spawns an isolate, runs the function, transfers the result, and tears down
  return await Isolate.run(() {
    final List<dynamic> decodedList = jsonDecode(rawJson) as List<dynamic>;
    return decodedList
      .map((map) => EmployeeEntity.fromJson(map as Map<String, dynamic>))
      .toList();
  });
}
```

10. Senior Behavioral, Cross-Functional & Scenario Questions

Scenario 1: Working in an Existing Codebase with Technical Debt

- **Question:** "How do you approach a large existing codebase that has poor architecture, no tests, and legacy patterns?"
- **Answer Structure (STAR):**
 - **Assess & Map:** Avoid judgment or hasty rewrites. Map the core critical user journeys and dependencies first.
 - **Lock Down with Tests:** Introduce unit tests around the existing behavior to establish a safety net.
 - **Modularize Incrementally:** Apply the Strangler Fig pattern. When tasked with modifying a feature, extract repository interfaces and isolate data sources.
 - **Establish Team Standards:** Set up static analysis rules (`analysis_options.yaml`) with lint rules agreed upon with the team so new code doesn't inherit old anti-patterns.

Scenario 2: Disagreement with Backend Team on API Schema

- **Question:** "The backend team provided an API response format that is poorly structured, sluggish to parse on mobile, and missing essential fields. How do you handle this?"
- **Answer Structure:**
 - Communicate with empathy and facts: Quantify the mobile impact (e.g., payload size, multiple round-trips causing high latency on low-bandwidth field connections).
 - Propose a concrete solution (e.g., an aggregated BFF / Backend-for-Frontend endpoint or GraphQL/RPC query).
 - In the interim: Write an Adapter / Data Transfer Object (DTO) in the Flutter data layer that maps their response cleanly into our pure domain entities so the presentation layer remains decoupled.

Scenario 3: Production Outage / Crash Triage Protocol

- **Question:** "A new release goes live and crash reports suddenly spike on Sentry / Firebase Crashlytics. What is your exact step-by-step reaction?"
- **Answer Steps:**
 1. **Triage & Containment:** Check the blast radius (which OS version, app version, or device model is affected). If critical and impacting financial transactions or core auth, immediately halt staged rollout on Google Play / App Store.
 2. **Reproduce & Root-Cause:** Pull the stack trace and breadcrumbs from Crashlytics. Match the build SHA with git tags and reproduce locally.
 3. **Hotfix or Rollback:** If a quick hotfix can be deployed within minutes, create a dedicated hotfix branch off the release tag, verify with regression tests, and submit an expedited review. If backend-related, toggle the remote feature flag.
 4. **Post-Mortem:** Document root cause, why the issue wasn't caught in automated tests or staging QA, and add a test case to prevent future regressions.

11. Reverse Questions to Ask the Interviewers

Asking intelligent questions proves your seniority and engineering maturity:

1. *"What does the current architecture of this internal application look like, and what are the main technical hurdles you're addressing ahead of the production launch?"*
2. *"How does the team currently handle testing and CI/CD between the mobile and portal/tablet release channels?"*
3. *"What is the balance right now between paying down legacy technical debt versus delivering new roadmap features?"*
4. *"How closely do the mobile engineers work with the backend, design, and QA teams during sprint cycles?"*